HARI SHANKAR — PERSONAL PORTFOLIO

MASTER AI SOFTWARE DEVELOPMENT PROMPT
Complete Product Specification + Technical Architecture + Data Flow + Integrity & Verification Protocol

Document status: Final master specification. This document is intended to be given directly to an AI coding agent. It defines both what the portfolio must become and how the AI must safely build, modify, test, document, and maintain it.

Core principle: The AI must build the actual working product—not merely generate attractive code or a visual prototype. The database, authentication, storage, public website, Admin system, caching, error handling, and deployment integrity are all part of the product.

Hallucination safeguard: No prompt can mathematically guarantee zero hallucinations. This specification therefore requires evidence-based development: inspect the actual repository, inspect the real database/configuration, never invent missing facts, verify changes, and explicitly report anything that could not be verified.

PART I — PRODUCT VISION

1. PROJECT PURPOSE

Build a premium personal portfolio website for Hari Shankar that presents his education, skills, experience, projects, certifications, resume, and contact information in a cinematic, technically sophisticated, professional interface.

This is not intended to be a static HTML portfolio. It is a real database-driven portfolio platform with a protected Admin system. Public visitors view content; the owner manages content through Admin without editing source code.

The final experience should communicate a combination of software engineering, technical creativity, modern product design, and professional credibility.

2. PRIMARY PRODUCT REQUIREMENTS

- Premium cinematic visual identity.
- Original design; do not produce a generic AI-generated portfolio template.
- Dark/light theme support with intentionally designed visuals for both.
- Engineering/technical 3D background language.
- Database-driven content.
- Protected Admin content management.
- Reliable persistence and database connectivity.
- Public pages update when Admin content changes.
- Projects support detailed case studies.
- Certifications support manual entry and file storage; OCR is NOT required now.
- Resume is a Home-page section, not a separate Resume route.
- Shared navigation and footer across public pages.
- Responsive desktop/tablet/mobile design.
- Accessibility and SEO.
- Strong local development and production integrity.
- Persistent human-readable project progress documentation.
- The system must remain understandable when the AI model is changed.

3. PRODUCT SUCCESS DEFINITION

The project succeeds only when visual quality and engineering integrity exist together. A beautiful frontend with fake data or broken persistence is a failure. A technically correct backend with an unfinished interface is also incomplete.

SUCCESS =
Visual quality
+ Correct user flows
+ Real database persistence
+ Secure Admin
+ Reliable storage
+ Correct cache/revalidation
+ Error handling
+ Responsive behavior
+ Accessibility
+ SEO
+ Verified build/runtime integrity
+ Maintainable documentation

PART II — TECH STACK AND SYSTEM ARCHITECTURE

4. PREFERRED TECHNOLOGY STACK

Use the following stack for a new implementation unless the existing partially developed project already uses a sound architecture that should reasonably be preserved.

FRONTEND / FULL-STACK FRAMEWORK
Next.js + TypeScript

UI / STYLING
Tailwind CSS

UI ANIMATION
Framer Motion

3D
Three.js + React Three Fiber
Use only where genuine 3D is required.

DATABASE
Supabase PostgreSQL

AUTHENTICATION
Supabase Auth

FILE STORAGE
Supabase Storage

HOSTING
Vercel

SOURCE CONTROL
Git + GitHub

Do not add libraries simply because they are popular. Before introducing a dependency, determine whether it is actually needed and whether the existing stack already provides the capability.

5. STACK RESPONSIBILITIES

Next.js + TypeScript

- Application framework, routing, server/client boundaries, server-side operations, metadata, and production build.

Tailwind CSS

- Consistent responsive styling, layout, spacing, responsive breakpoints, themes, and component styling.

Framer Motion

- Page transitions, reveal animations, scroll-triggered UI motion, staggered content, and micro-interactions.

Three.js / React Three Fiber

- Particle portrait, technical background elements, nodes, wireframes, particles, and other genuine 3D effects. Keep 3D isolated and performance-aware.

Supabase PostgreSQL

- Persistent portfolio content and relational data.

Supabase Auth

- Protected Admin authentication and session handling.

Supabase Storage

- Resume, certificates, thumbnails, project images, and other controlled uploaded assets.

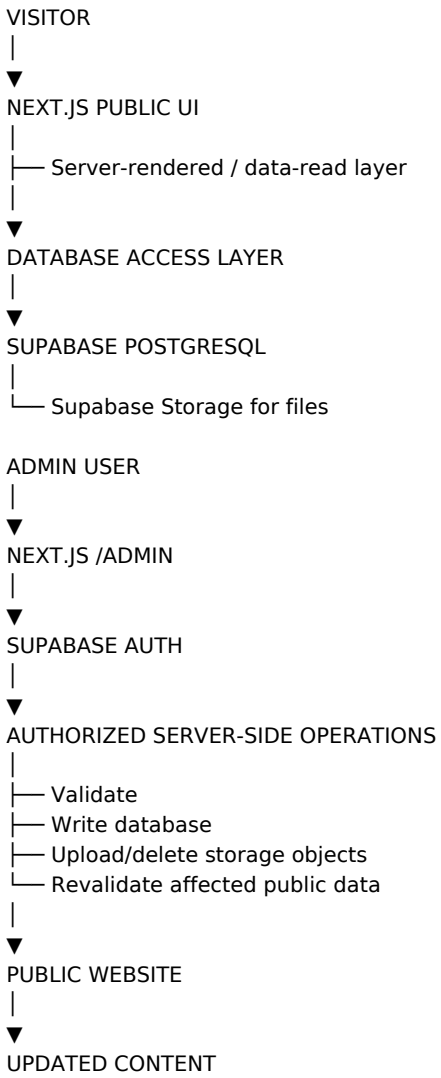
Vercel

- Production hosting and deployment.

Git/GitHub

- Version control, history, rollback, and collaboration.

6. HIGH-LEVEL SYSTEM ARCHITECTURE



7. SECURITY BOUNDARY

Never expose privileged database credentials or Supabase service-role credentials in browser/client code. Public pages should only receive data they are intended to display. Admin mutations must pass authentication and authorization checks server-side.

Hiding an Admin button is not authorization. The backend/server-side operation must independently enforce access.

PART III — INFORMATION ARCHITECTURE

8. PUBLIC ROUTES

```
/
/projects
/projects/[slug]
/skills
/experience
/education
/certifications
/contact
```

There is no separate /resume page.

9. PROTECTED ROUTES

```
/admin
/admin/...
```

The exact nested Admin routes may be chosen based on implementation, but all administrative operations must be protected.

10. SHARED SITE SHELL

- Shared header/navigation.
- Shared footer.
- Global theme handling.
- Global typography and spacing system.
- Global accessibility behavior.
- Consistent loading/error/not-found patterns.
- Consistent visual language.

Home should not appear as a standard navigation item because it is the landing page itself. On mobile, use a hamburger menu in the left corner as specified.

11. HOME PAGE — CINEMATIC OVERVIEW

1. Cinematic introduction
2. Hero/profile introduction
3. Short About preview
4. Highlighted skills
5. Featured projects
6. Experience preview
7. Education preview
8. Featured/latest certifications
9. Resume download section
10. Contact CTA
11. Shared footer

Home is a curated trailer/overview, not a dump of every database record. Featured flags, ordering, and/or limits should determine what appears.

12. CINEMATIC INTRO — EXACT CONCEPT

```
BOOTING
MY PORTFOLIO
```

- Dark premium environment.
- Sequential refined letter reveal.
- No cheap terminal blinking effect.
- Short hold after the phrase completes.

- Letters dissolve into hundreds/thousands of particles.
- Particles flow and gather to form a portrait based on the uploaded profile image.
- Particle portrait transitions smoothly into the actual profile image.
- Transition into the Hero section.
- Provide reduced-motion and low-performance fallbacks.
- The animation must never block access to the website.

13. VISUAL WORLD

Create one coherent visual language based on engineering and digital systems:

- Floating nodes.
- Circuit-like connections.
- Wireframe structures.
- Geometric forms.
- Particles.
- Subtle data streams.
- Technical/mathematical patterns.
- Depth and parallax.
- Glassmorphism where appropriate.

The background should always feel subtly alive, but content must remain readable and visually dominant.

14. DARK/LIGHT MODE

Dark and light themes must be deliberately designed rather than simply inverting colors.

- Check contrast in both themes.
- Ensure glass surfaces remain legible.
- Ensure 3D background remains subtle in both themes.
- Do not create unreadable text or invisible controls.
- Persist the user's theme choice using an appropriate mechanism.

PART IV — DATABASE DESIGN

15. DATABASE AS THE SINGLE CONTENT SOURCE

Portfolio content that is intended to be editable in Admin must come from the database. Do not hard-code such content into public components.

The exact SQL schema may be adapted after inspecting the existing project, but the conceptual model should support the following.

16. CORE DATA ENTITIES

profile

- id
- name
- headline
- profile_image_url
- short_bio
- location (only if intentionally published)
- contact information as appropriate
- created_at
- updated_at

about

- id
- content
- display settings as needed
- updated_at

skills

- id
- category
- name
- description (optional)
- display_order
- published
- created_at
- updated_at

projects

- id
- title
- slug
- short_description
- full_description
- problem_goal
- solution
- features
- technical_stack
- challenges
- learnings
- architecture_notes
- github_url
- live_url (nullable)
- featured
- display_order
- status (draft/published)
- created_at
- updated_at

project_images

- id
- project_id

storage_path / public URL strategy
alt_text
display_order
created_at

experience

id
organization
role
start_date
end_date
description
responsibilities
technologies
display_order
status
created_at
updated_at

education

id
institution
program
start_date
end_date
academic_information
description
display_order
status
created_at
updated_at

certifications

id
title
issuer
description
completion_date
credential_id (nullable)
file_storage_path
thumbnail_storage_path (nullable)
featured
display_order
status
created_at
updated_at

resume / site_settings

Resume metadata and global settings may be modeled separately or within a site_settings/content table depending on the implementation. The design must allow a resume file to be replaced without editing source code.

social_links

id
platform
label
url
display_order
published

17. DATABASE RELATIONSHIPS

```
projects
│
└──< project_images

profile
about
skills
experience
education
certifications
social_links
site_settings / resume
│
└── independent content entities
```

Use proper foreign keys and constraints where appropriate. Avoid duplicating the same content in multiple places merely for convenience.

18. DATABASE MIGRATION SAFETY

- Inspect current schema before migration.
- Never assume a table is unused.
- Do not casually drop tables.
- Preserve existing records wherever possible.
- Use migrations or controlled schema changes.
- After migration, verify the application can still read existing content.
- Update PROJECT_PROGRESS.txt with database changes.

19. DATABASE CRUD CONTRACT

Every Admin-managed entity must have a predictable lifecycle:

```
FORM INPUT
→ CLIENT-FRIENDLY VALIDATION
→ SERVER-SIDE VALIDATION
→ AUTHORIZATION
→ DATABASE/STORAGE OPERATION
→ VERIFY RESULT
→ RETURN SUCCESS/ERROR
→ REVALIDATE PUBLIC DATA
→ UI REFLECTS RESULT
```

Never rely only on client-side validation. The server-side operation must independently validate and authorize the request.

PART V — ADMIN SYSTEM

20. ADMIN PURPOSE

Admin is the private control center for managing portfolio content without editing source files.

The Admin interface should be understandable even to a non-programmer. Forms should use clear labels, helpful descriptions, validation messages, confirmation states, and safe delete/update flows.

21. ADMIN CONTENT MANAGEMENT

- Profile
- About
- Skills
- Projects
- Experience
- Education
- Certifications
- Resume
- Social links
- Site settings

22. PROJECT ADMIN FLOW

Create/Edit Project
→ Enter information
→ Validate
→ Upload images if required
→ Save database record
→ Save image metadata
→ Confirm successful save
→ Revalidate affected pages
→ Verify public project
→ Record progress

If Live Demo is not supplied, the public page must not display an empty Live Demo control.

23. CERTIFICATION ADMIN FLOW

Enter certificate information
→ Select file
→ Validate file type/size
→ Upload to controlled storage
→ Save metadata to database
→ Confirm storage + database success
→ Revalidate certification pages
→ Verify public view

OCR/AI extraction is explicitly out of scope for the current version.

24. RESUME ADMIN FLOW

Select replacement PDF
→ Validate
→ Upload controlled storage
→ Save/replace metadata
→ Confirm
→ Revalidate Home
→ Verify download

25. ADMIN DELETE SAFETY

For destructive operations, provide an intentional confirmation step. Consider whether dependent records or storage objects must also be handled. Do not delete a database record while leaving uncontrolled orphaned storage files unless that is an intentional documented decision.

PART VI — PUBLIC CONTENT AND UX

26. PROJECT PUBLIC PAGE

Project cards should provide concise information. Detail pages should provide the complete story:

- Project overview.
- Problem/goal.
- Solution.
- Features.
- Technical stack.
- Architecture/technical notes.
- Challenges.
- Learnings.
- GitHub.
- Live Demo only when available.

27. CERTIFICATION PUBLIC PAGE

Display certificate title, issuer, date, description, credential ID where applicable, thumbnail/preview, and a safe way to view the certificate file. Public users cannot modify records.

28. EXPERIENCE PUBLIC PAGE

Present experiences using a clean timeline/card layout. Avoid a plain copied resume-table appearance.

29. EDUCATION PUBLIC PAGE

Present academic history using timeline/cards with useful academic details.

30. SKILLS PUBLIC PAGE

Group skills by meaningful categories. Avoid fake numerical proficiency unless there is a legitimate source.

31. CONTACT

Contact should provide reliable contact methods and social links configured through the database where appropriate. Do not render broken/empty links.

PART VII — TECHNICAL DATA FLOW AND INTEGRITY

32. PUBLIC READ FLOW

Browser
↓
Next.js Route/Page
↓
Server-side data access
↓
Supabase query
↓
Validated result
↓
UI rendering
↓
Visitor sees content

For public content, use the appropriate server/client boundary. Do not unnecessarily expose privileged database access to the browser.

33. ADMIN WRITE FLOW

Admin Browser
↓
Authenticated session
↓
Protected Admin route
↓
Server-side authorization
↓
Input validation
↓
Database / Storage mutation
↓
Verify returned result
↓
Revalidate affected route/data
↓
Return explicit success/error
↓
Admin UI updates
↓
Public UI reads new data

34. FILE STORAGE FLOW

Admin selects file
↓
Client validation for UX
↓
Server-side validation / authorized upload
↓
Supabase Storage
↓
Store metadata/path in database
↓
Verify upload
↓
Public page receives safe file URL/reference

The database should hold the metadata/reference needed to identify the stored asset; do not treat a browser-only file selection as successful persistence.

35. CACHE / REVALIDATION FLOW

Admin mutation succeeds
↓
Determine affected public routes
↓
Invalidate/revalidate appropriate cached data
↓
Next public request reads updated state
↓
Verify with refresh/direct navigation

Do not solve stale content by indiscriminately disabling all caching. Use deliberate revalidation based on the framework and data lifecycle.

36. FAILURE FLOW

Request
↓
Failure detected
↓
Do NOT claim success
↓
Capture useful error information
↓
Show understandable user-facing error
↓
Offer retry when appropriate
↓
Log/document technical cause
↓
Fix root cause
↓
Retest

37. DATABASE INTEGRITY CHECKLIST

- Can the application connect?
- Can public pages read records?
- Can Admin create records?
- Can Admin update records?
- Can Admin delete records safely?
- Do invalid inputs fail correctly?
- Do changes persist after refresh?
- Do public pages reflect changes?
- Are relationships intact?
- Are storage records consistent with database references?
- Does authorization prevent unauthorized mutations?
- Are database failures surfaced rather than hidden?

PART VIII — AI DEVELOPMENT INTEGRITY

38. INSPECT BEFORE IMPLEMENT

The AI must inspect the existing project before making significant changes. This is mandatory for both new and existing projects.

```
INSPECT:
- repository tree
- package.json
- framework/config
- environment example
- routes
- pages
- components
- server/API logic
- database client
- schema/migrations
- auth
- storage
- styles
- tests
- README
- PROJECT_PROGRESS.txt
- ARCHITECTURE.md
- PROJECT_DECISIONS.md
```

39. NO INVENTED FACTS

Never invent:

- A file that was not found.
- A database table that was not verified.
- A successful test that was not run.
- A database connection that was not tested.
- A package that was not installed.
- An API response that was not received.
- A deployment that was not verified.
- A user-provided requirement that was never given.
- Credentials or secrets.

40. EVIDENCE LEVELS

When reporting project status, distinguish:

```
VERIFIED
Directly inspected or tested.

INFERRED
Strongly suggested by available evidence but not directly tested.

UNKNOWN / NOT VERIFIED
Cannot currently be confirmed.

BLOCKED
Cannot be tested or implemented because a required dependency, credential, service, or decision is unavailable.
```

Do not report INFERRED or UNKNOWN as VERIFIED.

41. MODEL SWITCHING

A new AI model must begin by inspecting the repository and reading the persistent documentation. It must not assume that the previous model's conversation is available.

```
NEW MODEL
→ Read README.md
→ Read PROJECT_PROGRESS.txt
→ Read ARCHITECTURE.md
→ Read PROJECT_DECISIONS.md
→ Inspect actual source code
→ Inspect package/configuration
→ Verify critical claims
→ Continue from real state
```

If documentation disagrees with code, code/runtime evidence wins. Document the discrepancy and correct the documentation.

42. AMENDMENT INTEGRITY

When I request an amendment, preserve the requirement exactly while integrating it into the existing architecture. Do not silently reinterpret a clear request.

If the requested change affects database schema, authentication, storage, caching, or multiple public routes, analyze all affected dependencies before editing.

43. CHANGE IMPACT ANALYSIS

Before a significant change, identify:

- What will change.
- What depends on it.
- Which database entities are affected.
- Which public routes are affected.
- Which Admin screens are affected.
- Which cache/revalidation paths are affected.
- Which tests need to run.

This prevents a small amendment from silently breaking unrelated parts of the portfolio.

PART IX — QUALITY, TESTING, AND LOCALHOST INTEGRITY

44. TESTING LAYERS

Use the appropriate level of verification for each change:

- Static checks — lint/type checking/build.
- Unit tests — isolated logic where useful.
- Integration tests — database/API/auth interactions where useful.
- Browser/UI tests — important user journeys.
- Manual verification — visual and responsive behavior.
- Production build verification — final runtime integrity.

45. REQUIRED CORE USER JOURNEYS

- Visitor opens Home.
- Visitor navigates to Projects.
- Visitor opens a project detail page.
- Visitor opens Skills.
- Visitor opens Experience.
- Visitor opens Education.
- Visitor opens Certifications.
- Visitor downloads resume.
- Visitor opens Contact links.
- Admin logs in.
- Admin creates/edits a project.
- Public project reflects the change.
- Admin uploads a certification.
- Public certification reflects the change.
- Admin replaces resume.
- Public resume download reflects the replacement.
- Admin logs out.
- Unauthorized visitor cannot access protected Admin operations.

46. LOCAL DEVELOPMENT CHECK

When the project is run locally, it must not have avoidable startup/runtime errors.

Inspect package.json and run the actual available commands. Typical commands may include:

```
npm install
npm run lint
npm run typecheck
npm run build
npm run start
```

Do not assume these scripts exist. Use the project's real scripts.

47. PRODUCTION READINESS GATE

Do not call the project deployment-ready until:

- Production build succeeds.
- Production server starts.
- Important routes work.
- Direct URL refresh works.

- Database operations work.
- Admin authorization works.
- Storage works.
- Important forms work.
- No critical application-caused console errors remain.
- No broken images/links remain in tested areas.
- Responsive behavior is verified.
- Documentation is updated.
- PROJECT_PROGRESS.txt accurately describes reality.

PART X — RESPONSIVE, ACCESSIBILITY, SEO, PERFORMANCE

48. RESPONSIVE DESIGN

- Desktop.
- Laptop.
- Tablet.
- Mobile.

Do not simply shrink desktop. Use appropriate mobile layouts. Preserve the visual identity while reducing expensive 3D/animation complexity on weaker devices.

49. ACCESSIBILITY

- Semantic HTML.
- Keyboard navigation.
- Visible focus.
- Accessible labels.
- Alt text.
- Correct heading hierarchy.
- Contrast in dark and light themes.
- Reduced-motion support.
- Accessible mobile navigation.
- Accessible modal/lightbox behavior.

50. SEO

- Page-specific title.
- Page-specific description.
- Open Graph metadata.
- Canonical URLs.
- Sitemap.
- Robots configuration.
- Clean project slugs.
- Useful structured data where appropriate.
- 404 page.

51. PERFORMANCE

- Optimize images.
- Lazy-load non-critical assets where appropriate.
- Avoid unnecessary client components.
- Avoid unnecessary network requests.
- Optimize database queries.
- Avoid excessive 3D objects.
- Use reduced complexity on mobile/low-power devices.
- Do not sacrifice correctness for a superficial performance score.

PART XI — DOCUMENTATION AND MODEL HANDOFF

52. README.md

Must explain:

- What the project is.
- Technology stack.
- Installation.
- Environment variables.
- Development commands.
- Production build/run commands.
- High-level architecture.
- Database setup.
- How Admin works.
- How to deploy.
- Important warnings.

53. ARCHITECTURE.md

Must explain the current real architecture, not an imagined architecture.

- Frontend structure.
- Route structure.
- Server/client boundaries.
- Database access.
- Authentication.
- Storage.
- Admin flow.
- Public data flow.
- Cache/revalidation.
- Important dependencies.

54. PROJECT_DECISIONS.md

Record important decisions such as:

- Why the hybrid multi-page architecture was selected.
- Why database-driven content is used.
- Why OCR is currently excluded.
- Why resume is a Home section.
- Important schema decisions.
- Important authentication decisions.
- Important performance decisions.
- Important architectural trade-offs.

55. PROJECT_PROGRESS.txt

This is the non-programmer-friendly development diary and current-state handoff file. It must be updated after every meaningful work session.

PART XII — IMPLEMENTATION ROADMAP

56. PHASE 0 — PROJECT AUDIT

- Inspect existing project.
- Inspect stack.
- Inspect database.
- Inspect environment.
- Inspect current pages.
- Inspect current Admin.
- Inspect current errors.
- Create/update project documentation.
- Produce a gap analysis.

57. PHASE 1 — FOUNDATION

- Confirm framework.
- Confirm TypeScript.
- Confirm styling system.
- Confirm folder architecture.
- Confirm environment handling.
- Confirm Git hygiene.
- Confirm error boundaries/loading/not-found behavior.

58. PHASE 2 — DATABASE

- Connect/verify database.
- Inspect/create schema.
- Create safe migrations.
- Verify reads/writes.
- Configure storage.
- Verify persistence.

59. PHASE 3 — AUTH + ADMIN

- Authentication.
- Authorization.
- Admin shell.
- Admin navigation.
- Admin CRUD.
- Validation.
- Safe destructive operations.

60. PHASE 4 — PUBLIC CONTENT

- Home.
- Projects.
- Project details.
- Skills.
- Experience.
- Education.

- Certifications.
- Contact.
- Resume section.

61. PHASE 5 — CINEMATIC EXPERIENCE

- Cinematic boot sequence.
- Particle portrait.
- Hero transition.
- Technical 3D environment.
- Scroll animation.
- Theme refinement.
- Performance fallbacks.

62. PHASE 6 — INTEGRATION

- Admin → database.
- Database → public pages.
- Storage → public pages.
- Cache/revalidation.
- Error/loading states.
- Regression testing.

63. PHASE 7 — QUALITY GATE

- Lint.
- Typecheck.
- Build.
- Start production server.
- Browser testing.
- Responsive testing.
- Database testing.
- Auth testing.
- Storage testing.
- Security review.
- SEO/accessibility review.
- Documentation review.

PART XIII — MANDATORY WORK SESSION PROCEDURE

64. BEFORE WORK

A. Read the user's latest request.
B. Read relevant project documentation.
C. Inspect actual implementation.
D. Identify affected systems.
E. Identify risks.
F. Make a concise implementation plan.

65. DURING WORK

A. Make focused changes.
B. Preserve unrelated working code.
C. Verify database/storage operations.
D. Avoid assumptions.
E. Keep changes understandable.
F. If a new problem appears, diagnose it rather than hiding it.

66. AFTER WORK

A. Run relevant tests.
B. Run broader checks if the change is significant.
C. Verify the actual user journey.
D. Check for regressions.
E. Update documentation.
F. Update PROJECT_PROGRESS.txt.
G. Give the user a plain-language report.

67. USER REPORT FORMAT

WHAT YOU ASKED FOR
[plain-language summary]

WHAT I CHANGED
[plain-language summary]

WHAT IS NOW WORKING
[verified items]

DATABASE
[connected / not connected / what changed / verification]

TESTING
[PASS / FAIL / NOT VERIFIED]

PROBLEMS REMAINING
[honest list]

NEXT STEP
[next logical task]

68. WHEN BLOCKED

If the AI cannot continue because of missing credentials, unavailable services, unclear requirements, permissions, or another real blocker:

- Do not fabricate a solution.
- State exactly what is blocking progress.
- Explain the minimum action required from me.
- Continue with independent tasks when safe.

- Record the blocker in PROJECT_PROGRESS.txt.

69. FINAL DEFINITION OF DONE

PROJECT READY only when:

✓ Public pages work
✓ Navigation works
✓ Responsive layouts work
✓ Cinematic intro works or has a verified fallback
✓ Projects work
✓ Project detail pages work
✓ Skills work
✓ Experience works
✓ Education works
✓ Certifications work
✓ Resume download works
✓ Contact works
✓ Admin authentication works
✓ Admin authorization works
✓ Admin CRUD works
✓ Database persistence works
✓ Storage works
✓ Admin changes appear publicly
✓ Cache/revalidation works
✓ Error states work
✓ Loading states work
✓ Empty states work
✓ Critical console errors resolved
✓ Build succeeds
✓ Production runtime verified
✓ Security basics verified
✓ Accessibility reviewed
✓ SEO reviewed
✓ Documentation updated
✓ PROJECT_PROGRESS.txt is current

70. FINAL AI OPERATING COMMAND

Build the real product, not the appearance of a product.

Inspect before changing. Verify before claiming. Preserve what works. Fix root causes. Treat the database as a critical production system. Implement my amendments exactly. Keep public content and Admin content synchronized. Document every meaningful session. Make the project understandable to the next AI model. If something cannot be verified, say so.

The repository, database, configuration, and verified runtime behavior are reality. This prompt is the specification. Never reverse those priorities.

END OF HARI SHANKAR PORTFOLIO MASTER AI DEVELOPMENT PROMPT